Structural Intelligence — Key Terms and Acronyms
A Reader's Guide to the Structural Intelligence Research Landscape

Sizhe Tan
with chatGPT (OpenAI)
2026-08-16

Repository: https://github.com/sizhet/Structural-Intelligence-Compass

The **Structural Intelligence Compass** connects concepts developed across multiple Structural Intelligence research directions.
This glossary provides a compact entry point for readers encountering those terms for the first time.
It is not intended to replace the corresponding papers or repositories.
Its purpose is to answer three practical questions:

1. **What does this term mean?**
2. **What structural role does it play?**
3. **Where should I look next?**

1. Quick Reference Table

Acronym / Term	Full Name	Structural Role	Primary Research Direction
SI	Structural Intelligence	Umbrella research perspective emphasizing structure, organization, runtime computation, invariants, and structural boundaries	Structural Intelligence research family

SRMS	Structural Recognition above Metric Similarity	Distinguishes structural recognition from metric similarity	SRMS
CCC	Common Concept Core	Extracts or represents structure preserved across multiple instances	FTRIA / SRAI
Two-Way CCC	Two-Way Common Concept Core	Preserve–Subtract structural comparison between two structures	FTRIA / FTRIS / SRAI
Unaligned AND	Unaligned AND	Structural intersection without requiring positional alignment	FTRIA / SRAI
BTP	Bucket Tree of Permutations	Organizes permutation/ search structure for structural comparison	FTRIA / SRAI
FT	Function Tunnel	Reusable computational channel or runtime path	FTRIA / FTRIS / RCP
RI	Runtime Invariant	Structure preserved across runtime variation	FTRIA / RCP
RIA	Runtime Invariant Algebra	Algebraic treatment of Runtime Invariants and their composition	FTRIA

FTRIA	Function Tunnel and Runtime Invariant Algebra	Framework connecting Function Tunnels, Runtime Invariants, CCC, switching, and runtime computation	FTRIA
FTRI	Function-Tunnel / Runtime-Invariant structure	Runtime computational structure connecting Function Tunnels and Runtime Invariants	FTRIS / RCP
FTRIS	Structure and Dynamics of FTRI Switching	Study of runtime switching among FTRI structures	FTRIS
RCP	Runtime Computational Primitives	Research into minimal computational primitives for runtime structure and switching	RCP
MET	Minimal Evolution Threshold	Threshold for distinguishing meaningful structural evolution from incremental variation	SI research methodology
UTN	Universal Typing and Naming	Provides structural typing, naming, identity, and addressability	CKOI / CG

CKOI	Computational Knowledge Organization and Infrastructure	Organizes computational knowledge into reusable structural infrastructure	CKOI
GTDO	Goal-Oriented Training Data Organization	Organizes data and computational units according to runtime goals and RHS requirements	GTDO
PDOS	Policy-Driven Organizational Synthesis	Synthesizes computational organization through policy-driven structural projection	PDOS
PDOR	Policy-Driven Organizational Runtime	Runtime implementation direction for PDOS: organization, routing, local decision, feedback	PDOR
X–Y–M	State–Decision–Measure Memory	Local memory connecting state X, decision/ output Y, and measured result M	PDOS / PDOR
CG	Core-Preserved Generation	Generation that preserves a structural core while allowing controlled variation	SI / AI Coding

CPC / Calling-Path Compression	Calling-Path Compression	Compresses experienced computational paths into reusable runtime capability	SI / LLM structural interpretation
LNI	Local Node Intelligence	Intelligence executed locally within structurally addressed runtime nodes	PDOS / PDOR / Hybrid AI
WM	World Model	Model of objects, relations, transitions, dynamics, and possible future states	Hybrid Intelligence
BI	Boundary Intelligence	Ability to recognize, represent, test, communicate, and expand validity boundaries	SI Compass
Residual Intelligence	Residual Intelligence	Portion of an intelligent task not naturally explained by the current structural basis	SI Compass
Basis Stress Testing	Basis Stress Testing	Method for actively searching for stable residuals in the current structural basis	SI Compass

Operator Economy	Operator Economy	Preference for a small reusable operator basis over unnecessary primitive proliferation	FTRIA / SI
Hybrid Intelligence	Hybrid Intelligence	Runtime organization of heterogeneous intelligence mechanisms	SRAI / SI Compass

2. Structural Recognition and Comparison

SI — Structural Intelligence

Structural Intelligence is the umbrella perspective underlying this research family.

Its central concern is not only whether two objects are metrically similar or whether a model can generate a useful answer.

It asks how computation can identify, preserve, organize, route, compare, activate, and validate **structure**.

Typical SI questions include:

- What structure is preserved?
- What structure differs?
- Where does computation belong?
- What can call what?
- Which runtime path should become active?
- What must remain invariant?
- Where does the validity boundary end?

SRMS — Structural Recognition above Metric Similarity

Structural Recognition above Metric Similarity distinguishes:

metric closeness

from:

structural correspondence.

Two objects may be metrically close while differing in a critical structure.

Two objects may also be metrically distant while preserving the structure relevant to the task.

SRMS therefore provides an important foundation for:

- structural recognition;
- extrapolation;

- differential organization;
- structural confidence.

Follow-up direction: Structural Recognition above Metric Similarity (SRMS).

CCC — Common Concept Core

The **Common Concept Core** represents structure preserved across multiple instances.

Conceptually:

A

B

C

→ **Preserved Common Structure**

→ **CCC**

CCC can support:

- concept formation;
- structural recognition;
- invariant extraction;
- comparison;
- representation.

A central Structural Intelligence interpretation is:

Concepts can be treated computationally as preserved structure.

Follow-up direction: FTRIA and SRAI.

Two-Way CCC

Two-Way CCC extends structural comparison into two complementary directions:

Preserve

What remains common?

Subtract

What differs?

Conceptually:

Structure A

vs.

Structure B

→ **Preserved Structure**

- **Differential Structure**

This simple operation can support:

- recognition;
- comparison;
- trigger discovery;

- structural switching;
- multi-perspective analysis;
- X–Y–M differentiation.

Two-Way CCC is especially important in the Compass because it may provide a route from accumulated runtime experience to **generative trigger discovery**.
Follow-up direction: FTRIA, FTRIS, SRAI.

Unaligned AND

Unaligned AND performs structural intersection without requiring corresponding elements to occupy the same positional alignment.
It is useful when the same structural relation may appear in different positions, orders, or representations.
It therefore supports structural recognition beyond rigid positional matching.
Follow-up direction: FTRIA and SRAI.

BTP — Bucket Tree of Permutations

The **Bucket Tree of Permutations** organizes permutation-related search into a structured search space.
Its purpose is to reduce brute-force structural comparison and support efficient recognition under varying arrangement.
BTP is an example of a broader SI principle:
Organize combinatorial possibility before searching it.
Follow-up direction: FTRIA and SRAI.

3. Runtime Structural Computation

FT — Function Tunnel

A **Function Tunnel** is a reusable computational channel.
Conceptually:
Input / Condition
→ **Computational Tunnel**
→ **Output / Effect**
A Function Tunnel may represent:

- a learned computational path;
- an algorithmic transformation;
- a runtime behavior;
- a decision channel;
- a structural perspective.

Multiple Function Tunnels can provide multiple computational perspectives on the same problem.

Follow-up direction: FTRIA, FTRIS, RCP.

RI — Runtime Invariant

A **Runtime Invariant** is structure that remains preserved across runtime variation.

If a Function Tunnel changes state while some relationship remains stable, that stable relationship may be treated as an RI.

Runtime Invariants support:

- validation;
- structural preservation;
- switching;
- generative constraints;
- runtime reasoning.

Follow-up direction: FTRIA and RCP.

RIA — Runtime Invariant Algebra

Runtime Invariant Algebra studies Runtime Invariants as composable computational objects.

The long-term objective is to understand how preserved runtime structures may be:

- compared;
- composed;
- transformed;
- sequenced;
- switched.

A key Structural Intelligence observation is:

A Function Tunnel may be understood as a sequence or organization of Runtime Invariants.

Follow-up direction: FTRIA.

FTRIA — Function Tunnel and Runtime Invariant Algebra

FTRIA develops the relationship among:

- Function Tunnels;
- Runtime Invariants;
- CCC;
- Two-Way CCC;
- structural comparison;
- triggering;
- switching;
- runtime algebra.

It is one of the main theoretical foundations behind the runtime side of Structural Intelligence.

FTRI

FTRI refers to the combined structural view of Function Tunnels and Runtime Invariants as runtime computational structures.
The important transition is from:
static computational possibility
to:
runtime-selectable computational structure.

FTRIS — Structure and Dynamics of FTRI Switching

FTRIS studies what happens when runtime computation can switch among FTRI structures.
It introduces questions such as:

- What triggers a switch?
- What remains invariant?
- What changes?
- How does an Actor influence switching?
- How can competing runtime channels be compared?

Applications include:

- AI control;
- robotics;
- markets;
- games;
- multi-agent systems.

RCP — Runtime Computational Primitives

Runtime Computational Primitives asks whether structures such as:

- selective reachability;
- runtime authority;
- triggering;
- switching;
- Function Tunnels;
- Runtime Invariants

may represent a more general class of computational primitives.
RCP distinguishes:
Potential Structure
from:
Runtime Structure.

A system may contain many possible computational relationships while activating only a small subset at any particular moment.

4. Knowledge and Organizational Structures

UTN — Universal Typing and Naming

Universal Typing and Naming provides structural:

- typing;
- naming;
- identity;
- reference;
- addressability.

Conceptually:

Object
→ **Type**
→ **Name**
→ **Structural Position**
→ **Reusable Reference**

UTN is especially important for Core-Preserved Generation because a system cannot reliably preserve a core unless it can identify what structural object is being preserved.

UTN also supports:

- computational knowledge organization;
- graph construction;
- persistent identity;
- generation validation.

Follow-up direction: CKOI and Core-Preserved Generation.

CKOI — Computational Knowledge Organization and Infrastructure

CKOI studies how computational knowledge can be transformed into reusable organizational infrastructure.

Its central progression is:

Knowledge
→ **Representation**
→ **Typing / Naming**
→ **Organization**
→ **Computational Infrastructure**

CKOI provides an important bridge between representation and runtime organization.

GTDO — Goal-Oriented Training Data Organization

GTDO studies how data can be organized according to the computational goal it must support.
Rather than treating training data as one globally connected pool, GTDO emphasizes:

- grouping;
- dispatch;
- localized computational units;
- runtime isolation;
- structural security.

This contributes to the broader SI movement from:
global undifferentiated intelligence
toward:
organized and localized intelligence.

5. Policy-Driven Organization and Runtime

PDOS — Policy-Driven Organizational Synthesis

PDOS studies how structural organization can be projected into operational organization through policy.
A simplified pipeline is:
X-Space
→ **Differential Organization**
→ **Policy**
→ **Runtime Address**
The key idea is:
Organization can become computation.
PDOS connects knowledge organization with:

- routing;
- runtime decision making;
- local intelligence;
- feedback.

PDOR — Policy-Driven Organizational Runtime

PDOR is the runtime and engineering direction associated with PDOS.
A simplified PDOR loop is:
X
→ **Organizer**
→ **Policy Router**
→ **Local Node**
→ **Decision Y**

→ **Measure M**
→ **Feedback**
PDOR therefore attempts to make organizational intelligence executable.

X–Y–M

X–Y–M is a compact local runtime memory structure.

X

State, context, or observed condition.

Y

Decision, action, or output.

M

Measured effect or outcome.
Repeated observations produce:
X–Y–M history
→ **Local Decision Landscape**
This can support:

- local learning;
- policy improvement;
- validation;
- trigger discovery.

A particularly important future direction is:
Multi-Valued X–Y–M
→ **Two-Way CCC**
→ **Candidate Trigger**
→ **Runtime Validation**

6. Generative Structural Intelligence

CG — Core-Preserved Generation

Core-Preserved Generation describes generation in which some structural core must remain invariant while other structure may change.
Conceptually:
Preserved Core

- **Controlled Delta**

→ **Generated Result**
This is especially important for AI Coding.
A generator may modify:

- implementation;
- decomposition;
- optimization;

- naming;
- local algorithms;

while preserving:

- API contracts;
- required behavior;
- data semantics;
- architectural constraints;
- certified invariants.

CG therefore treats generation as:
controlled structural variation rather than unrestricted production.

Variable-Size Block Index

A **Variable-Size Block Index** provides structural addressing when meaningful computational units do not have fixed size.
This can support:

- hierarchical blocks;
- nested structures;
- variable-length code regions;
- adaptable structural scopes.

It complements UTN:
UTN
answers:
What structural object is this?
Variable-Size Block Index
helps answer:
What computational span belongs to this object?

7. Calling Structures and LLMs

Calling Graph

A **Calling Graph** represents potential computational reachability:
What can call what?
Nodes may represent:

- functions;
- models;
- tools;
- agents;
- local intelligence units;
- humans.

Edges represent possible computational relationships.
A Calling Graph is therefore a **potential runtime structure**.

The actual runtime system activates only selected paths.

Calling Path

A **Calling Path** is an active or realizable route through a Calling Graph.
Conceptually:
Node A
→ **Node B**
→ **Node C**
→ **Result**
Calling Paths connect static architecture to actual runtime computation.

Calling-Path Folding

Calling-Path Folding describes the structural compression of many experienced computational paths into a more compact reusable structure.

CPC — Calling-Path Compression

Calling-Path Compression can be represented as:
Large Experience Space
→ **Many Experienced Calling Paths**
→ **Folded Representation**
→ **Runtime Reconstruction**
This provides one Structural Intelligence perspective on Large Language Models.
It does **not** claim that an LLM is completely reducible to Calling-Path Compression.
Rather, it highlights one important structural property:
Large bodies of computational experience can be folded into broad reusable runtime capability.

8. Local and Hybrid Intelligence

LNI — Local Node Intelligence

Local Node Intelligence means that intelligence is executed within a structurally localized node rather than requiring one global intelligence mechanism to process every state.
A local node may use:

- LLM;
- Two-Way CCC;
- deterministic algorithm;
- probabilistic engine;
- causal model;

- simulator;
- human decision.

The architectural principle is:
Shared organization does not require shared intelligence implementation.

WM — World Model

A **World Model** represents objects, relations, dynamics, transitions, and possible future states.
Within the Structural Intelligence Compass, World Models are treated as complementary rather than competing with Structural Intelligence.
A future Hybrid Intelligence system may combine:
LLM

- **SI**
- **WM**
- **Causal Models**
- **Tools**
- **Humans**

through an organized runtime structure.

Hybrid Intelligence

Hybrid Intelligence refers here not merely to connecting multiple AI systems.
It means:
runtime organization across heterogeneous forms of intelligence.
Important questions include:

- Who receives runtime authority?
- What triggers each system?
- What can call what?
- Which perspective should be trusted?
- How are disagreements handled?
- How is the result validated?

9. Research Methodology

MET — Minimal Evolution Threshold

Minimal Evolution Threshold is used to distinguish a meaningful structural

evolution from a smaller quantitative change.
When evaluating a new mechanism, ask:
Does this represent a genuinely new structural step?
or:
Is it mainly scale, tuning, implementation, or composition?
MET helps prevent every improvement from being promoted into a new structural primitive.

Operator Economy

Operator Economy is the preference for explaining broad computational behavior with the smallest reusable set of structural operators that remains adequate.
The principle is:
Prefer composition of reusable structures over unnecessary proliferation of first-class primitives.
Operator Economy is not minimalism for its own sake.
It aims to improve:

- explanatory clarity;
- implementation;
- validation;
- reuse.

Basis Stress Testing

Once a compact candidate structural basis emerges, the research methodology changes.
Instead of asking:
What new primitive can we invent?
ask:
What intelligent task cannot be naturally constructed using the current basis?
This is **Basis Stress Testing**.
The basic loop is:
Construct
→ **Stress**
→ **Detect Residual**
→ **Classify**
→ **Revise if Necessary**
→ **Re-Test**

Residual Intelligence

Residual Intelligence is:
The portion of an intelligent task that cannot yet be naturally represented, organized, generated, routed, triggered, learned, executed, or validated using the current structural basis.
Possible residuals include:

- Representation Residual;
- Identity Residual;
- Segmentation Residual;
- Organizational Residual;
- Generative Residual;
- Experience Residual;
- Comparison Residual;
- Runtime Residual;
- Memory Residual;
- Validation Residual;
- Boundary Residual.

A stable residual that survives alternative decompositions and appears across multiple domains may justify investigation of a new first-class primitive.

Residual-Based Coverage Confidence

Residual-Based Coverage Confidence describes confidence in the coverage of a candidate structural basis.
The principle is:
Confidence in structural coverage increases when increasingly diverse stress tests fail to expose stable unexplained residuals.
But:
No Residual Found ≠ No Residual Exists
Therefore Residual-Based Coverage Confidence is not a completeness proof. It is an evidence-based confidence measure under the current scope of testing.

BI — Boundary Intelligence

Boundary Intelligence is:
The capability of an intelligent system to recognize, represent, test, communicate, and expand the validity boundaries of its own knowledge and computation.
A Boundary-Intelligent system should increasingly distinguish:

- known;
- uncertain;

- extrapolated;
- conflicting;
- out-of-domain;
- unknown.

The state **Unknown** can itself become a runtime trigger.

10. Three Major Compression Structures

The Structural Intelligence Compass currently identifies three especially broad candidate compression structures.

Major Structure	Compresses	Primary Question
Organizational Compression	Large state / structural spaces	**Where?**
Core-Preserved Generative Compression	Large generative spaces	**What must remain?**
Calling-Path Compression	Large experiential / computational path spaces	**What can be reused?**

These structures motivate the:

Minimal Structural Basis Hypothesis

Broad intelligence may require fewer first-class structural primitives than its behavioral complexity suggests.

This is a research hypothesis.
It is not a completeness claim.

11. Structural Intelligence Research Map

For readers who wish to continue beyond the Compass:

Research Direction	Primary Focus
SRMS	Structural recognition beyond metric similarity
FTRIA	CCC, Two-Way CCC, Function Tunnels, Runtime Invariants, runtime algebra
SRAI	Structural Runtime AI and integration of SI operators
GTDO	Goal-oriented data and computational organization

FTRIS	Runtime switching and FTRI dynamics
RCP	Runtime Computational Primitives
CKOI	Computational knowledge organization, UTN, and infrastructure
PDOS	Policy-Driven Organizational Synthesis
PDOR	Executable policy-driven organizational runtime
SI Compass	Cross-framework navigation, boundaries, basis convergence, and stress testing

12. Suggested Reading Paths

Readers do not need to follow the complete research history.
Different questions suggest different entry paths.

If you are interested in structural recognition

SRMS
→ **CCC**
→ **Two-Way CCC**
→ **SRAI**

If you are interested in runtime computation

FT
→ **RI**
→ **FTRIA**
→ **FTRIS**
→ **RCP**

If you are interested in knowledge organization

UTN
→ **CKOI**
→ **PDOS**
→ **PDOR**

If you are interested in AI Coding

UTN
→ **Core-Preserved Generation**
→ **Variable-Size Block Index**

→ **Calling Graph**
→ **Validation**

If you are interested in Digital Brain architecture

Differential Organization
→ **Calling Graph**
→ **PDOS / PDOR**
→ **Local Node Intelligence**
→ **FTRI Switching**
→ **Hybrid Intelligence**

If you are interested in research methodology

MET
→ **Operator Economy**
→ **Minimal Structural Basis**
→ **Basis Stress Testing**
→ **Residual Intelligence**
→ **Residual-Based Coverage Confidence**
→ **Boundary Intelligence**

13. A Compact Mental Model

A reader can remember much of the Structural Intelligence landscape through a small set of questions:

Question	Structural Mechanism
What is it?	UTN
What is common?	CCC
What is preserved and what differs?	Two-Way CCC
Where does it belong?	Differential Organization
What can call what?	Calling Graph
Which path should run?	FT / Triggering / Switching
What must remain stable?	RI / Core Preservation
What may change?	Controlled Delta / CG
What happened last time?	X–Y–M
What experience can be reused?	Calling-Path Compression
Did it work?	Measure / Feedback / Validation
Where does confidence end?	Boundary Intelligence
What are we still missing?	Residual Intelligence

14. Closing Guide

The acronyms in Structural Intelligence describe a connected research landscape rather than an isolated vocabulary.

A useful progression is:

Recognize Structure
→ **Name Structure**
→ **Organize Structure**
→ **Preserve Structure**
→ **Call Structure**
→ **Trigger Structure**
→ **Execute Locally**
→ **Measure**
→ **Validate**
→ **Detect the Boundary**
→ **Search for the Residual**

The purpose of this glossary is therefore not only to decode abbreviations.

It is to help readers move from:

What does this acronym mean?

to:

What structural problem does it solve?

and finally:

Where should I go next?

Structural Intelligence Compass

Term → Structural Role → Research Direction → Deeper Reading

Use the glossary as a road-sign system, not as a substitute for the road.